MEMBANGUN LIBRARY PROGRAM .H DAN .CPP

NRP	: 1. 3223600035
	2. 3223600040
	3. 3223600058
	4. 3223600060
Nama	: 1. Mohamad Kharizal Firdaus
	2. Rahmatulloh Syaifudin Fahmi
	3. Ihsanta Zaki Sanjaya
	4. Mochammad Ega Yudhistira
Kelas	: 2 D4 Teknik Komputer B
Mata Kuliah	: Praktikum Alat Pengembangan Perangkat Lunak
Tanggal	: Kamis, 24 April 2025
Dosen	: Dr. Setiawardhana, S.T., M.T.

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2025

MEMBANGUN LIBRARY PROGRAM .H DAN .CPP

I. Tujuan

1. Mahasiswa dapat mengembangkan atau membuat sendiri *.h sederhana
2. Mahasiswa dapat melakukan kompilasi ulang dari *.h

II. Dasar Teori

Library di dalam Arduino IDE adalah sekumpulan kode yang berisi fungsionalitas dan algoritma yang telah diprogram sebelumnya. Dengan adanya library, pengguna dapat memperkaya proyek Arduino mereka tanpa perlu menulis kode dari awal. Library memungkinkan kita untuk memanfaatkan kode yang sudah ada untuk menangani berbagai fungsi umum atau spesifik, seperti mengontrol motor servo, berkomunikasi dengan sensor, atau menghubungkan Arduino ke jaringan Wi-Fi. Alih-alih menulis kode yang kompleks dari nol, Anda hanya perlu memanggil fungsi-fungsi yang sudah ada di dalam library yang dipilih.

Pada dasarnya, library di Arduino terdiri dari beberapa komponen utama yang membantu Anda mengakses fungsionalitasnya dengan lebih mudah. Elemen-elemen tersebut adalah:

- **Header File (.h):** Merupakan file yang berisi deklarasi fungsi, variabel, dan struktur data yang disediakan oleh library. Header file ini perlu di-include dalam kode program utama Anda agar

dapat mengakses fungsionalitas yang ada di dalamnya.

- **Source File (.cpp):** Di dalam source file ini, implementasi dari fungsi-fungsi yang didefinisikan di header file ditulis. Source file ini dikompilasi bersamaan dengan kode program Arduino Anda untuk menghasilkan program yang bisa dijalankan.
- **Contoh Kode**: Banyak library yang disertai dengan contoh kode yang dapat membantu Anda memahami cara kerja dan penerapan fungsionalitas library tersebut dalam proyek Anda. Contoh kode ini sangat berguna bagi pemula untuk cepat beradaptasi dan mempelajari cara menggunakan library dengan benar.
- **Dokumentasi:** Setiap library biasanya dilengkapi dengan dokumentasi lengkap yang tersedia secara online, yang memuat penjelasan mendalam tentang fungsi, variabel, dan parameter yang ada dalam library tersebut. Dokumentasi ini sangat penting untuk memahami cara memanfaatkan library secara maksimal dan menghindari kesalahan dalam penggunaan.

Dalam praktikum ini, fokus pembuatan library ditujukan pada perangkat LCD (Liquid Crystal Display). LCD (Liquid Crystal Display) adalah salah satu komponen output yang umum digunakan untuk menampilkan data dalam bentuk teks dari mikrokontroler. Untuk mengendalikan LCD secara

langsung, programmer perlu mengetahui berbagai instruksi yang digunakan, seperti membersihkan tampilan, mengatur posisi kursor, dan menulis karakter. Dalam praktikum ini mahasiswa diarahkan untuk membuat library sendiri sebagai bentuk pemahaman lebih dalam terhadap cara kerja library dan proses komunikasi antara Arduino dan LCD. Dengan membuat library secara mandiri, mahasiswa dapat memahami proses inisialisasi, pengiriman data, serta pengaturan posisi kursor pada LCD, yang biasanya tersembunyi di balik fungsi siap pakai dari library bawaan.

III. Alat dan Bahan

1. Arduino UNO
2. Kabel USB Arduino
3. Liquid Crystal Display (LCD) 16x2
4. Laptop

IV. Prosedur Percobaan

1. Buat file header (.h) untuk mendefinisikan class, fungsi, variabel, dan struktur yang akan digunakan.

```
#ifndef ObjectLcd_h
#define ObjectLcd_h

#include <inttypes.h>
#include "Print.h"

#define LCD_BERSIHKAN 0x01
#define LCD_KEMBALI_AWAL 0x02
#define LCD_MODE_ENTRI 0x04
#define LCD_KONTROL_TAMPILAN 0x08
#define LCD_GESER_CURSOR 0x10
#define LCD_SET_FUNGSI 0x20
#define LCD_SET_CGRAM 0x40
#define LCD_SET_DDRAM 0x80

#define LCD_ENTRI_KANAN 0x00
#define LCD_ENTRI_KIRI 0x02
#define LCD_AUTOGESER 0x01
#define LCD_TIDAK_AUTOGESER 0x00

#define LCD_TAMPILKAN 0x04
#define LCD_SEMBUNYIKAN 0x00
#define LCD_CURSOR_NYALA 0x02
#define LCD_CURSOR_MATI 0x00
#define LCD_KEDIP 0x01
#define LCD_TIDAK_KEDIP 0x00

#define LCD_MODE8BIT 0x10
#define LCD_MODE4BIT 0x00
#define LCD_2BARIS 0x08
#define LCD_1BARIS 0x00
#define LCD_5x10 0x04
#define LCD_5x8 0x00
```

```
class LayarLCD : public Print {
public:
  LayarLCD(uint8_t rs, uint8_t enable,
           uint8_t d0, uint8_t d1, uint8_t d2, uint8_t
d3,
           uint8_t d4, uint8_t d5, uint8_t d6, uint8_t
d7);

  void mulai(uint8_t kolom, uint8_t baris,
uint8_t ukuran_karakter = LCD_5x8);
  void bersihkan();
  void keAwal();
  void tampilkan();
  void sembunyikan();
  void aturPosisi(uint8_t kolom, uint8_t baris);
  virtual size_t tulis(uint8_t);

  using Print::write;

private:
  void kirim(uint8_t, uint8_t);
  void tulis4bit(uint8_t);
  void tulis8bit(uint8_t);
  void pulsaEnable();

  uint8_t _pin_rs;
  uint8_t _pin_rw;
  uint8_t _pin_enable;
  uint8_t _pin_data[8];

  uint8_t _fungsi_tampilan;
  uint8_t _kontrol_tampilan;
  uint8_t _mode_tampilan;
  uint8_t _baris;
  uint8_t _offset_baris[4];
};

#endif
```

2. Buat file implementasi (.cpp) untuk mengimplementasikan semua fungsi yang dideklarasikan di .h.

```
#include "ObjectLcd.h"
#include <Arduino.h>

LayarLCD::LayarLCD(uint8_t rs, uint8_t
enable,
                uint8_t d0, uint8_t d1, uint8_t d2,
uint8_t d3,
                uint8_t d4, uint8_t d5, uint8_t d6,
uint8_t d7) {
  _pin_rs = rs;
  _pin_rw = 255;
  _pin_enable = enable;

  _pin_data[0] = d0;
  _pin_data[1] = d1;
  _pin_data[2] = d2;
  _pin_data[3] = d3;
  _pin_data[4] = d4;
  _pin_data[5] = d5;
  _pin_data[6] = d6;
  _pin_data[7] = d7;

  _fungsi_tampilan = LCD_MODE8BIT |
LCD_1BARIS | LCD_5x8;
}

void LayarLCD::mulai(uint8_t kolom, uint8_t
baris, uint8_t ukuran_karakter) {
  if (baris > 1) {
    _fungsi_tampilan |= LCD_2BARIS;
  }
  _baris = baris;
  _offset_baris[0] = 0x00;
  _offset_baris[1] = 0x40;
  _offset_baris[2] = 0x00 + kolom;
  _offset_baris[3] = 0x40 + kolom;
```

```
if ((ukuran_karakter != LCD_5x8) && (baris
== 1)) {
    _fungsi_tampilan |= LCD_5x10;
  }

  pinMode(_pin_rs, OUTPUT);
  if (_pin_rw != 255) pinMode(_pin_rw,
OUTPUT);
  pinMode(_pin_enable, OUTPUT);

  for (int i = 0; i < 8; ++i) {
    pinMode(_pin_data[i], OUTPUT);
  }

  delayMicroseconds(50000);
  digitalWrite(_pin_rs, LOW);
  digitalWrite(_pin_enable, LOW);
  if (_pin_rw != 255) digitalWrite(_pin_rw,
LOW);

  kirim(LCD_SET_FUNGSI | _fungsi_tampilan,
LOW);
  delayMicroseconds(4500);
  kirim(LCD_SET_FUNGSI | _fungsi_tampilan,
LOW);
  delayMicroseconds(150);
  kirim(LCD_SET_FUNGSI | _fungsi_tampilan,
LOW);

  _kontrol_tampilan = LCD_TAMPILKAN;
  tampilkan();
  bersihkan();

  _mode_tampilan = LCD_ENTRI_KIRI |
LCD_TIDAK_AUTOGESER;
  kirim(LCD_MODE_ENTRI |
_mode_tampilan, LOW);
}

void LayarLCD::bersihkan() {
  kirim(LCD_BERSIHKAN, LOW);
  delayMicroseconds(2000);
}
```

```
void LayarLCD::keAwal() {
  kirim(LCD_KEMBALI_AWAL, LOW);
  delayMicroseconds(2000);
}

void LayarLCD::aturPosisi(uint8_t kolom,
uint8_t baris) {
  if (baris >= _baris) baris = _baris - 1;
  kirim(LCD_SET_DDRAM | (kolom +
_offset_baris[baris]), LOW);
}

void LayarLCD::tampilkan() {
  _kontrol_tampilan |= LCD_TAMPILKAN;
  kirim(LCD_KONTROL_TAMPILAN |
_kontrol_tampilan, LOW);
}

void LayarLCD::sembunyikan() {
  _kontrol_tampilan &= ~LCD_TAMPILKAN;
  kirim(LCD_KONTROL_TAMPILAN |
_kontrol_tampilan, LOW);
}

inline size_t LayarLCD::tulis(uint8_t karakter) {
  kirim(karakter, HIGH);
  return 1;
}

void LayarLCD::kirim(uint8_t nilai, uint8_t
mode) {
  digitalWrite(_pin_rs, mode);
  if (_pin_rw != 255) digitalWrite(_pin_rw,
LOW);

  for (int i = 0; i < 8; i++) {
    digitalWrite(_pin_data[i], (nilai >> i) &
0x01);
  }
  pulsaEnable();
}
```

```
void LayarLCD::pulsaEnable() {
  digitalWrite(_pin_enable, LOW);
  delayMicroseconds(1);
  digitalWrite(_pin_enable, HIGH);
  delayMicroseconds(1);
  digitalWrite(_pin_enable, LOW);
  delayMicroseconds(100);
}
```

3. Buat file .ino untuk memanggil dan menggunakan class/fungsi dari file .h dan .cpp.

```
#include "ObjectLcd.h"

LayarLCD lcd(12, 11, 5, 4, 3, 2, 6, 7, 8, 9);

void setup() {
  lcd.mulai(16, 2);
  lcd.aturPosisi(0, 0);
  lcd.print("Hello LCD!");
}

void loop() {
}
```

4. Buat folder dengan nama sketch_apr27b. Di dalam folder ini dibuat sub-folder dengan nama ObjectLcd, yang nantinya menjadi direktori library.
5. Letakkan file ObjectLcd.h dan ObjectLcd.cpp yang telah dibuat sebelumnya ke dalam folder ObjectLcd.
6. Di dalam folder ObjectLcd, buat folder bernama examples. Di dalamnya buat lagi folder bernama Hello, dan di dalam folder Hello dibuat file

Hello.ino yang berisi contoh program penggunaan library ObjectLcd.

7. Setelah seluruh struktur folder selesai, folder ObjectLcd dikompres (di-zip) menjadi file ObjectLcd.zip untuk proses instalasi ke dalam Arduino IDE.
8. Tambahkan Library ke Arduino IDE
    a. Buka Arduino IDE
    b. Pilih menu Sketch → Include Library → Add .ZIP Library...
    c. Pilih file ObjectLcd.zip yang telah dibuat
    d. Akan muncul notifikasi "Successfully installed library from ObjectLcd.zip archive"
    e. Tutup dan buka kembali Arduino IDE
    f. Pilih menu File → Examples → ObjectLcd → Hello
    g. Setelah file Hello.ino terbuka, lakukan proses compile dan upload ke board Arduino.

V. Hasil Percobaan

1. File .h

```
1   #ifndef ObjectLcd_h
2   #define ObjectLcd_h
3
4   #include <inttypes.h>
5   #include "Print.h"
6
7   #define LCD_BERSIHKAN 0x01
8   #define LCD_KEMBALI_AWAL 0x02
9   #define LCD_MODE_ENTRI 0x04
10  #define LCD_KONTROL_TAMPILAN 0x08
11  #define LCD_GESER_CURSOR 0x10
12  #define LCD_SET_FUNGSI 0x20
13  #define LCD_SET_CGRAM 0x40
14  #define LCD_SET_DDRAM 0x80
15
16  #define LCD_ENTRI_KANAN 0x00
17  #define LCD_ENTRI_KIRI 0x02
18  #define LCD_AUTOGESER 0x01
19  #define LCD_TIDAK_AUTOGESER 0x00
20
21  #define LCD_TAMPILKAN 0x04
22  #define LCD_SEMBUNYIKAN 0x00
23  #define LCD_CURSOR_NYALA 0x02
24  #define LCD_CURSOR_MATI 0x00
25  #define LCD_KEDIP 0x01
26  #define LCD_TIDAK_KEDIP 0x00
27
28  #define LCD_MODE8BIT 0x10
29  #define LCD_MODE4BIT 0x00
30  #define LCD_2BARIS 0x08
31  #define LCD_1BARIS 0x00
32  #define LCD_5x10 0x04
33  #define LCD_5x8 0x00
34
35  class LayarLCD : public Print {
36  public:
37    LayarLCD(uint8_t rs, uint8_t enable,
38             uint8_t d0, uint8_t d1, uint8_t d2, uint8_t d3,
39             uint8_t d4, uint8_t d5, uint8_t d6, uint8_t d7);
40
41    void mulai(uint8_t kolom, uint8_t baris, uint8_t ukuran_karakter = LCD_5x8);
42    void bersihkan();
43    void keAwal();
44    void tampilkan();
45    void sembunyikan();
46    void aturPosisi(uint8_t kolom, uint8_t baris);
47    virtual size_t tulis(uint8_t);
48
49    using Print::write;
50
51  private:
52    void kirim(uint8_t, uint8_t);
53    void tulis4bit(uint8_t);
54    void tulis8bit(uint8_t);
55    void pulsaEnable();
56
57    uint8_t _pin_rs;
58    uint8_t _pin_rw;
59    uint8_t _pin_enable;
60    uint8_t _pin_data[8];
61
62    uint8_t _fungsi_tampilan;
```

```
  uint8_t _kontrol_tampilan;
  uint8_t _mode_tampilan;
  uint8_t _baris;
  uint8_t _offset_baris[4];
};

#endifSSS
```

2. File .cpp

```
#include "ObjectLcd.h"
#include <Arduino.h>

LayarLCD::LayarLCD(uint8_t rs, uint8_t enable,
                   uint8_t d0, uint8_t d1, uint8_t d2, uint8_t d3,
                   uint8_t d4, uint8_t d5, uint8_t d6, uint8_t d7) {
  _pin_rs = rs;
  _pin_rw = 255;
  _pin_enable = enable;

  _pin_data[0] = d0;
  _pin_data[1] = d1;
  _pin_data[2] = d2;
  _pin_data[3] = d3;
  _pin_data[4] = d4;
  _pin_data[5] = d5;
  _pin_data[6] = d6;
  _pin_data[7] = d7;

  _fungsi_tampilan = LCD_MODE8BIT | LCD_1BARIS | LCD_5x8;
}

void LayarLCD::mulai(uint8_t kolom, uint8_t baris, uint8_t ukuran_karakter) {
  if (baris > 1) {
    _fungsi_tampilan |= LCD_2BARIS;
  }
  _baris = baris;
  _offset_baris[0] = 0x00;
  _offset_baris[1] = 0x40;
  _offset_baris[2] = 0x00 + kolom;
  _offset_baris[3] = 0x40 + kolom;

  if ((ukuran_karakter != LCD_5x8) && (baris == 1)) {
    _fungsi_tampilan |= LCD_5x10;
  }

  pinMode(_pin_rs, OUTPUT);
  if (_pin_rw != 255) pinMode(_pin_rw, OUTPUT);
  pinMode(_pin_enable, OUTPUT);

  for (int i = 0; i < 8; ++i) {
    pinMode(_pin_data[i], OUTPUT);
  }

  delayMicroseconds(50000);
  digitalWrite(_pin_rs, LOW);
  digitalWrite(_pin_enable, LOW);
  if (_pin_rw != 255) digitalWrite(_pin_rw, LOW);

  kirim(LCD_SET_FUNGSI | _fungsi_tampilan, LOW);
  delayMicroseconds(4500);
  kirim(LCD_SET_FUNGSI | _fungsi_tampilan, LOW);
  delayMicroseconds(150);
  kirim(LCD_SET_FUNGSI | _fungsi_tampilan, LOW);

  _kontrol_tampilan = LCD_TAMPILKAN;
  tampilkan();
  bersihkan();

  _mode_tampilan = LCD_ENTRI_KIRI | LCD_TIDAK_AUTOGESER;
  kirim(LCD_MODE_ENTRI | _mode_tampilan, LOW);
}
```

```
void LayarLCD::bersihkan() {
  kirim(LCD_BERSIHKAN, LOW);
  delayMicroseconds(2000);
}

void LayarLCD::keAwal() {
  kirim(LCD_KEMBALI_AWAL, LOW);
  delayMicroseconds(2000);
}

void LayarLCD::aturPosisi(uint8_t kolom, uint8_t baris) {
  if (baris >= _baris) baris = _baris - 1;
  kirim(LCD_SET_DDRAM | (kolom + _offset_baris[baris]), LOW);
}

void LayarLCD::tampilkan() {
  _kontrol_tampilan |= LCD_TAMPILKAN;
  kirim(LCD_KONTROL_TAMPILAN | _kontrol_tampilan, LOW);
}

void LayarLCD::sembunyikan() {
  _kontrol_tampilan &= ~LCD_TAMPILKAN;
  kirim(LCD_KONTROL_TAMPILAN | _kontrol_tampilan, LOW);
}

inline size_t LayarLCD::tulis(uint8_t karakter) {
  kirim(karakter, HIGH);
  return 1;
}

void LayarLCD::kirim(uint8_t nilai, uint8_t mode) {
  digitalWrite(_pin_rs, mode);
  if (_pin_rw != 255) digitalWrite(_pin_rw, LOW);

  for (int i = 0; i < 8; i++) {
    digitalWrite(_pin_data[i], (nilai >> i) & 0x01);
  }
  pulsaEnable();
}

void LayarLCD::pulsaEnable() {
  digitalWrite(_pin_enable, LOW);
  delayMicroseconds(1);
  digitalWrite(_pin_enable, HIGH);
  delayMicroseconds(1);
  digitalWrite(_pin_enable, LOW);
  delayMicroseconds(100);
}
```

3. File .ino

```
#include "ObjectLcd.h"

LayarLCD lcd(12, 11, 5, 4, 3, 2, 6, 7, 8, 9);

void setup() {
  lcd.mulai(16, 2);
  lcd.aturPosisi(0, 0);
  lcd.print("Hello LCD!");
}

void loop() {
}
```

4. Output LCD

VI. Analisa

Pada praktikum ini, mahasiswa membangun sebuah library sendiri untuk mengontrol LCD menggunakan pendekatan pemrograman berorientasi objek di Arduino. Proses ini melibatkan tiga bagian utama yaitu file header (.h), file implementasi (.cpp), dan file program utama (.ino).

Pada file ObjectLcd.h, dibuat sebuah class bernama LayarLCD yang berisi deklarasi fungsi-fungsi penting seperti mulai, aturPosisi, tampilkan, bersihkan, dan sebagainya. File ini juga mendeklarasikan variabel-variabel private untuk menyimpan pin-pin yang digunakan, mode tampilan, serta konfigurasi karakter dan baris LCD. Adanya pendeklarasian class ini bertujuan agar pengguna cukup memanggil fungsi-fungsi yang tersedia tanpa perlu mengetahui detail teknis komunikasi ke LCD secara langsung.

Selanjutnya, file ObjectLcd.cpp merupakan bagian yang mengimplementasikan semua fungsi yang sudah dideklarasikan pada header. Misalnya, fungsi mulai() digunakan untuk inisialisasi LCD dan konfigurasi mode tampilan 8-bit. Fungsi kirim() bertugas mengirimkan data maupun perintah ke LCD dengan mengatur pin-pin digital sesuai data yang dikirimkan. Fungsi tulis() menampilkan karakter di layar, sedangkan aturPosisi() mengatur posisi kursor. Semua proses pengiriman data dikendalikan dengan fungsi pulsaEnable() yang memberikan pulsa ke pin enable LCD. Dengan mengelompokkan logika ini dalam satu file, kode menjadi lebih bersih dan mudah dipahami.

Terakhir, file utama ino adalah program yang memanfaatkan library yang telah dibuat. Di dalamnya, objek LayarLCD dibuat dengan menyebutkan semua pin yang digunakan. Pada fungsi setup(), dilakukan inisialisasi LCD dengan ukuran 16x2, kemudian kursor diarahkan ke posisi awal (baris pertama, kolom pertama), lalu menampilkan tulisan "Hello LCD!". Fungsi loop() dibiarkan kosong karena tidak ada proses yang berjalan terus-menerus dalam program ini.

Hasil dari praktikum ini menunjukkan bahwa library yang dibangun berhasil bekerja dengan baik. Pesan "Hello LCD!" berhasil tampil di layar LCD sesuai instruksi. Ini membuktikan bahwa semua fungsi dalam class LayarLCD telah terintegrasi dengan benar.

VII. Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa dengan membangun library sendiri, mahasiswa dapat lebih memahami cara kerja komunikasi antara mikrokontroler dan perangkat eksternal seperti LCD. Pembuatan library menggunakan struktur file .h untuk deklarasi, .cpp untuk implementasi, dan .ino untuk penggunaan, menjadikan program lebih modular. Library yang dibuat berhasil menampilkan teks "Hello LCD!" di layar, yang menunjukkan bahwa fungsi-fungsi dalam class LayarLCD telah berjalan dengan baik.